

Code-Layout, Readability and Reusability

Date	02 October 2026
Team ID	SWTID-2026-8542
Project Name	ComicCraft - AI Comic Story Creator using Gemini Models
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the supplied ComicCraft codebase in terms of its structure, readability, and reusability. The project document describes a modular architecture with separate functions and files for outline generation, story generation, image generation, layout building, PDF export, FastAPI routing and Jinja2 templates.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Code is organized into separate functions, routes and modules.	Yes	Modular project structure is documented.
2	Proper File Structure	Separate files are used for Gemini, image generation, layout, export and routes.	Yes	Examples include gemini_flash.py, gemini_pro.py, image_generator.py, layout_builder.py and exporters.py.
3	Meaningful Variable Names	Functions and routes clearly describe their purpose.	Yes	Examples: generate_outline(), generate_story(), generate_image(), save_pdf().
4	Function / Method Names	Functions are descriptively named according to their tasks.	Yes	Documented function names directly match the workflow.
5	Code Comments	The project documentation explains modules, activities and implementation steps.	Partial	Documentation is detailed; source-code comment coverage is not explicitly verified.
6	Modular Design	Core functions are separated into reusable modules.	Yes	The architecture separates frontend, backend and AI integration.
7	No Redundant Code	Separate components are assigned distinct responsibilities.	Partial	The supplied document does not provide enough source code to verify all redundancy.
8	Error Handling	Routes use HTTPException and try-except blocks for unexpected failures.	Yes	Explicitly documented for API and JSON processing errors.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	generate_outline()	Python / Gemini Flash	Comic panel outline generation	High
2	generate_story()	Python / Gemini Pro	Comic narration and dialogue generation	High

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
3	generate_image()	Python / Diffusers / Stable Diffusion	Comic-style image generation for panels	High
4	build_comic_layout()	Python	Combines panel images and story text	High
5	save_pdf()	Python / FPDF	Creates downloadable comic PDF	High

Code-Layout, Readability and Reusability

Reusable Components / Modules (continued):

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
6	FastAPI route handlers	Python / FastAPI	/generate, /generate-comic/json and related routes	High
7	Jinja2Templates	Python / Jinja2	index.html, comic_preview.html and export_success.html	High
8	PromptRequest	Python / Pydantic	Validation of JSON input for API-based generation	Medium
9	PDF export workflow	Python / FPDF	Converts panel layout and narration into a downloadable PDF	High
10	Virtual environment / requirements setup	Python / pip	Project setup and local deployment	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure		Modular architecture is documented, with separate components for AI generation, routing, layout and export.
Readability		Function and route names clearly communicate their purpose; the supplied documentation is organized by milestone and activity.
Reusability		Core generation, layout and export functions are separated and can be reused across the workflow.
Documentation / Comments		Project documentation is detailed. Actual source-code comment coverage is not explicitly verified from the supplied document.
Overall Score		Rating intentionally left blank for evaluator/team assessment.

Assessment note: The entries above are based on the supplied ComicCraft project documentation. Where the document did not provide enough evidence—such as exact source-code comment coverage or complete redundancy analysis—the checklist is marked Partial rather than assuming a result.